Assignment-13.3

Name: K.Nandini Reddy

HallTicket No:2303A51896

Batch: 08

Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance

Task 1: Refactoring – Removing Code Duplication

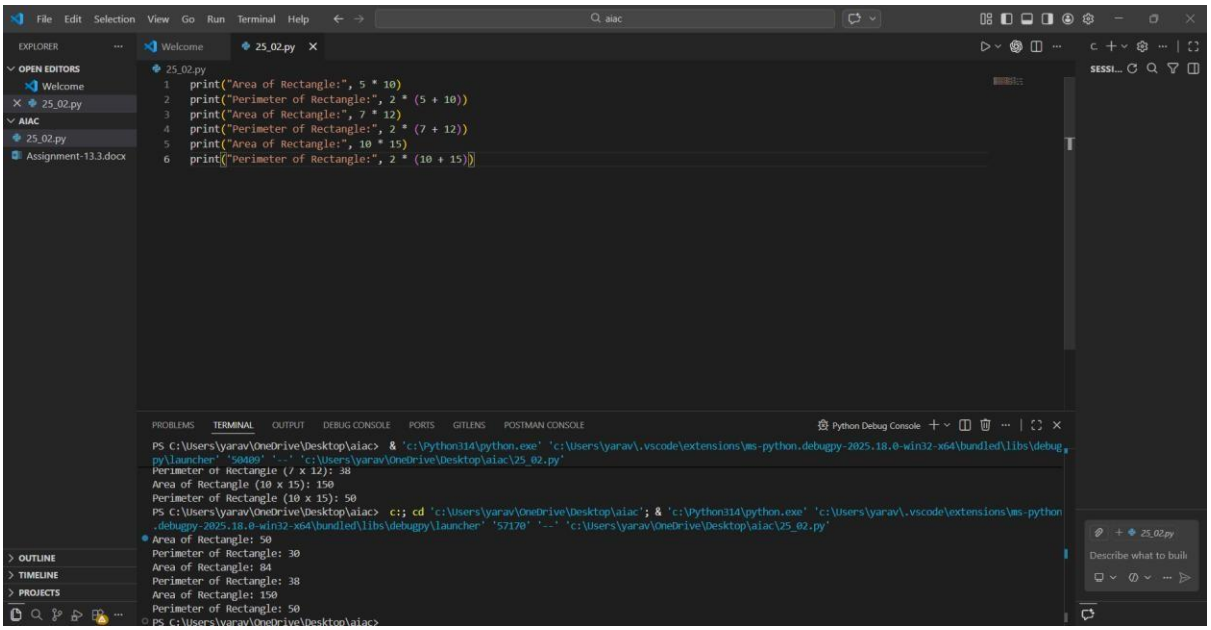
Objective

To eliminate repeated logic by extracting reusable functions.

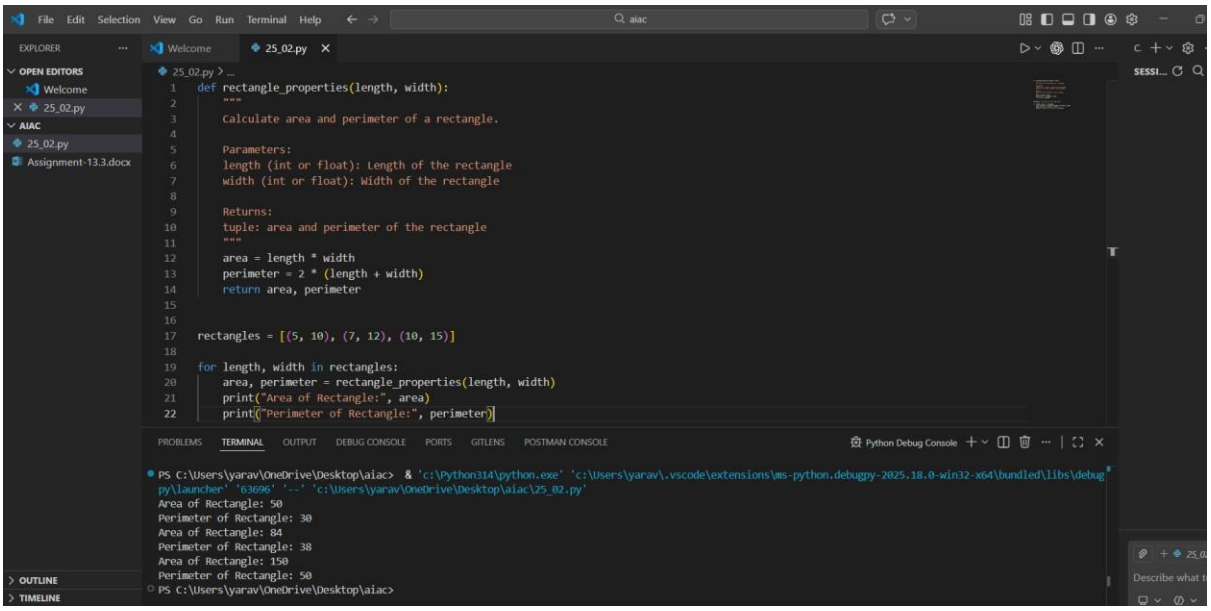
Prompt Used

“Refactor the following Python code to remove duplication and create reusable functions with proper docstrings.”

Legacy Code



Refactored Code

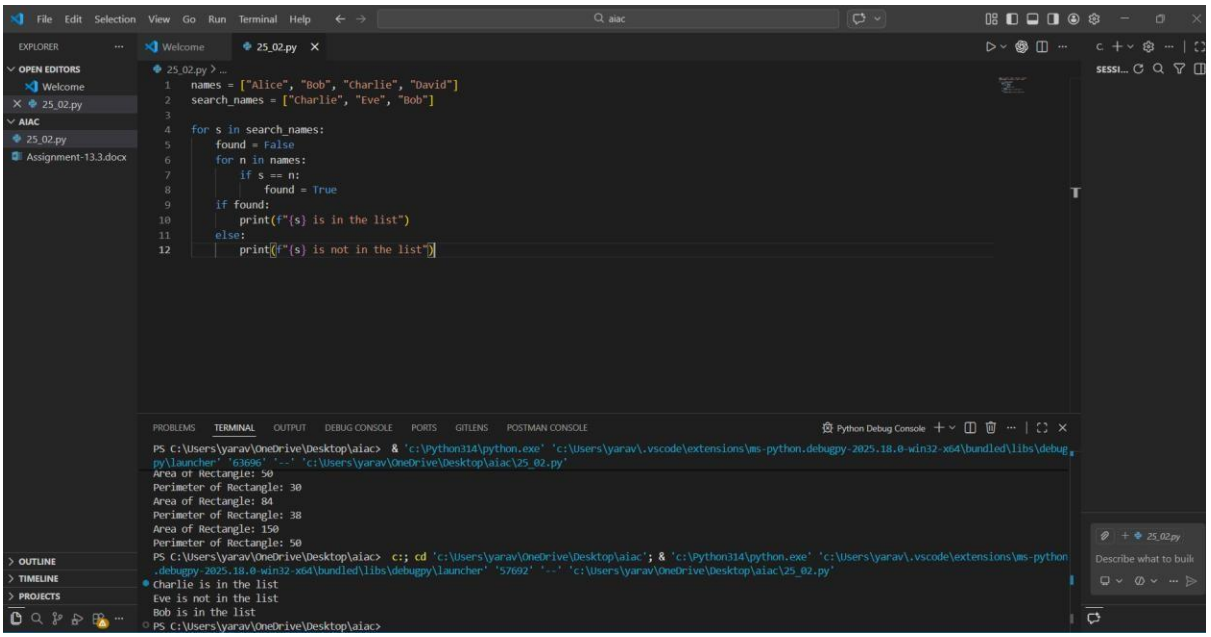


Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

Improve performance by replacing nested loops.

Legacy Code



Refactored Code (Using Set Lookup)

```python
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

name_set = set(names)  # O(1) lookup

for name in search_names:
    if name in name_set:
        print(f"{name} is in the list")
    else:
        print(f"{name} is not in the list")
```

```
PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs'
py\launcher' '57716' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_02.py'
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Task 3: Extracting Reusable Functions

Objective

Modularize price and tax calculations.

Legacy Code

```python
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Output:

```
PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debug
py\launcher' '54938' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_tas3'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Refactored Code

```python
TAX_RATE = 0.18

def calculate_total(price):
    """
    Calculate total price including tax.

    Parameters:
    price (float): Base price of the product

    Returns:
    float: Total price including tax
    """
    tax = price * TAX_RATE
    return price + tax


prices = [250, 500]

for price in prices:
    total = calculate_total(price)
    print("Total Price:", total)
```

Output:

```
PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\d
py\launcher' '49174' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_tas3'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Task 4: Replacing Hardcoded Values with Constants

Objective

Replace magic numbers with named constants.

Legacy Code

```python
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Output:

```
PROBLEMS   TERMINAL   OUTPUT   DEBUG CONSOLE   PORTS   GITLENS   POSTMAN CONSOLE        Python Debug Console

PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundl
py\launcher' '62574' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_task4.py'
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Refactored Code

```
Ass-13.3.py > ...
PI = 3.14159
RADIUS = 7

area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

Output:

```
PROBLEMS   TERMINAL   OUTPUT   DEBUG CONSOLE   PORTS   GITLENS   POSTMAN CONSOLE        Python Debug Console

PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundl
py\launcher' '62574' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_task4.py'
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Task 5: Improving Variable Naming and Readability

Objective

Use descriptive variable names.

Legacy Code

```
Ass-13.3.py > ...
a = 10
b = 20
c = a * b / 2
print(c)
```

Output:

```
PROBLEMS    TERMINAL    OUTPUT    DEBUG CONSOLE    PORTS    GITLENS    POSTMAN CONSOLE

PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-p
py\launcher' '49307' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_task5.py'
100.0
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Refactored Code

```
Ass-13.3.py > ...
base = 10
height = 20

# Calculate area of a triangle
triangle_area = (base * height) / 2

print(triangle_area)
```

Output:

```
PROBLEMS    TERMINAL    OUTPUT    DEBUG CONSOLE    PORTS    GITLENS    POSTMAN CONSOLE

PS C:\Users\yarav\OneDrive\Desktop\aiac>  & 'c:\Python314\python.exe' 'c:\Users\yarav\.vscode\extensions\ms-p
py\launcher' '49307' '--' 'c:\Users\yarav\OneDrive\Desktop\aiac\25_task5.py'
100.0
PS C:\Users\yarav\OneDrive\Desktop\aiac>
```

Conclusion

In this lab, AI-assisted refactoring significantly improved the quality of legacy Python code. Code duplication was removed, inefficient loops were optimized, reusable functions were created, magic numbers were replaced with constants, and meaningful variable names were introduced. The refactored programs are now more readable, maintainable, scalable, and performance-efficient while preserving original functionality.